Project Description: Rental Management Application

Project Overview:

A Rental Management Application is a comprehensive software solution developed to facilitate the management of rental properties, ensuring that property owners, property managers, and tenants can effectively and efficiently handle all aspects of the rental process. The application serves as a centralized platform for various tasks, including property listing, tenant onboarding, lease management, maintenance requests, and financial transactions.

Key Features and Functionalities:

1. Property Listings:
    - Property owners can create detailed listings of their rental properties, including property descriptions, rental rates, and high-quality photos.
    - Property managers can easily organize and promote available properties for rent.
2. Tenant Management:
    - Property managers can manage tenant information, including tenant applications, references, and lease agreements.
    - Tenants can submit applications, review lease terms, and communicate with property managers.
3. Lease Tracking:
    - The application helps property managers and property owners keep track of lease agreements, including lease terms, renewal dates, and rent payments.
4. Maintenance Requests:
    - Tenants can submit maintenance requests and track their status.
    - Property managers can assign and manage maintenance tasks efficiently.
5. Financial Management:
    - The application enables property owners and managers to handle financial transactions, including rent collection.
6. Communication and Messaging:
    - The application includes communication tools for property managers, property owners, and tenants to exchange messages and notifications.
7. Document Management:
    - Users can store and manage essential documents such as lease agreements, rental applications, and property inspection reports.
8. Reporting and Analytics:
    - The application provides insights into property performance, financial data, occupancy rates, and maintenance history through reports and analytics.
9. Security and Data Protection:
    - Robust security measures ensure the safety of sensitive data, including personal and financial information.

Group Project week 2

I. Source code management (20%)

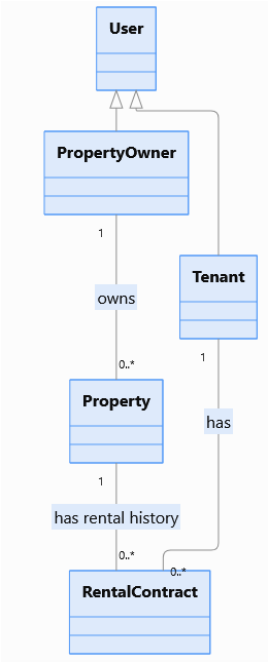
1. Create a "**GitHub repository**" as your main (root) directory.
2. Publish your "**Local repository**" to GitHub.
3. Create one new "**User story**" and title "Github deployment"
4. Add link to your GitHub repository inside "Github deployment".
5. Create 5 new "**User story**" based on 5 class in appendix session.
6. Split 5 class implementation between each group members.
7. Each member creates a separate branch from the main (root).
8. After completing their implementation, issue pull requests to owner.
9. Repository owner merge each branch into main.

II. Make sure that your board mirrors your current progress. (40%)

*Hint: Remember the purpose of a Kanban board.

1. What is a User story?
2. What do the columns of a Kanban board refer to? (or: What do the columns on a Kanban board represent?)
3. What have you done until this exercise, and how can you show your progress to your teacher?

Appendix



1. Basic concepts

Concept	Meaning
Class	A blueprint, such as Property.
Object	One instance, such as Apartment A101.
Attribute	Information stored in an object, such as RentalRate.
Method	An action an object performs, such as UpdateRentalRate().
Constructor	Initializes an object when it is created.
Inheritance	A child class receives attributes and methods from a parent class.

2. User class

Purpose: store information shared by owners and tenants. PropertyOwner and Tenant inherit from User; do not declare the inherited attributes again.

Technical task:

- Complete define all required Attribute.
- Implement Constructor.
- Implement all methods/ function.

Attribute	Type	Meaning
UserID	Integer	Unique user identifier
FirstName	String	First name
LastName	String	Last name
Email	String	Email address

Methods

1. User(userID, firstName, lastName, email) -> Constructor: assign the supplied values to the corresponding attributes.
2. GetFullName() → String: return FirstName and LastName separated by a space.
3. UpdateProfile(firstName, lastName, email) → None: update the profile; keep UserID unchanged.

3. PropertyOwner class

Purpose: represent a user who owns properties. Inherits all User attributes and methods.

Technical task:

- Complete define all required Attribute.
- Implement Constructor.
- Implement all methods/ function.

Additional attribute	Type	Initial value
OwnedProperties	List of Property objects	Empty list

Methods

1. PropertyOwner(userID, firstName, lastName, email) -> Constructor: call the User constructor, then initialize OwnedProperties as an empty list.
2. CreateProperty(propertyID, description, rentalRate) → Property: create a Property object, add it to OwnedProperties, and return the object. If construction fails, do not add anything to the list.
3. GetProperties() → List of Property objects: return the owner's properties.
4. OwnedProperties stores objects, not just IDs or descriptions. For example, [property1, property2] contains two Property objects.

4. Property Class

Purpose: represent one property that can be rented. Create properties through PropertyOwner.CreateProperty() so they are registered with their owner.

Technical task:

- Complete define all required Attribute.
- Implement Constructor.
- Implement all methods/ function.

Attribute	Type	Meaning or initial value
PropertyID	Integer	Unique property identifier
Description	String	Property description
RentalRate	Decimal number	Monthly rental price
IsAvailable	Boolean	Initially true

Methods

1. Property(propertyID, description, rentalRate) -> Constructor: check that rentalRate is greater than zero, assign the supplied values, and set IsAvailable to true. Raise an error if the rate is invalid.
2. UpdateDescription(description) → None: replace the description.
3. UpdateRentalRate(rentalRate) → None: check that the new rate is greater than zero, then update RentalRate. If invalid, raise an error and keep the old rate.

5. Tenant class

Purpose: represent a user who rents properties. Inherits all User attributes and methods.

Technical task:

- Complete define all required Attribute.
- Implement Constructor.
- Implement all methods/ function.

Additional attribute	Type	Initial value
RentalContracts	List of RentalContract objects	Empty list

Methods

1. Tenant(userID, firstName, lastName, email) -> Constructor: call the User constructor, then initialize RentalContracts as an empty list.
2. RentProperty(contractID, property, startDate, endDate) → RentalContract: create a contract using this tenant and the supplied Property object. Add it to RentalContracts and return it. If creation fails, leave the list unchanged.
3. GetRentalContracts() → List of RentalContract objects: return all contracts, including terminated contracts.

6. RentalContract

Purpose: connect one Tenant object to one Property object and manage the rental.

Technical task:

- Complete define all required Attribute.
- Implement Constructor.
- Implement all methods/ function.

Attribute	Type	Meaning or initial value
ContractID	Integer	Unique contract identifier
Property	Property object	The rented property
Tenant	Tenant object	The tenant renting it
StartDate	Date	Rental start date
EndDate	Date	Rental end date
RentAmount	Decimal number	Agreed monthly rent
IsActive	Boolean	Initially true

RentalContract(contractID, property, tenant, startDate, endDate)

- Check that property.IsAvailable is true and endDate is later than startDate.
- If either check fails, raise an error before changing any state.
- Copy property.RentalRate into RentAmount; set IsActive to true.
- Set property.IsAvailable to false.

Method

1. Terminate() → None: if the contract is active, set IsActive to false and Property.IsAvailable to true. If already inactive, do nothing.
2. Dates describe the rental period. For this simplified exercise, availability depends only on whether a contract is active, not on today's date.

7. Relationships and method logic

The examples below are pseudocode. Adapt the syntax to your programming language. "This tenant" means the Tenant object whose method is being called.

PropertyOwner CreateProperty

```
CreateProperty(propertyID, description, rentalRate):
    property = new Property(propertyID, description, rentalRate)
    add property to OwnedProperties
    return property
```

Tenant RentProperty

```
RentProperty(contractID, property, startDate, endDate):
    contract = new RentalContract(
        contractID, property, this tenant, startDate, endDate
    )
    add contract to RentalContracts
    return contract
```

The RentalContract constructor checks availability and dates. If it raises an error, execution must stop before adding the contract to the list.

RentalContract Terminate

```
Terminate():
    if IsActive is false:
        return
    IsActive = false
    Property.IsAvailable = true
```

The inactive check matters: an old contract must not make a property available after that property has been rented again under a new contract.

8 Example program and expected results

```
owner = new PropertyOwner(
    1, "Anna", "Nguyen", "anna@example.com"
)
tenant = new Tenant(
    2, "Minh", "Tran", "minh@example.com"
)
property = owner.CreateProperty(
    101, "Apartment A101, two bedrooms", 5000
)
contract = tenant.RentProperty(
    1001, property,
    Date(2026, 11, 1), Date(2027, 10, 31)
)
```

Expression	Expected value
owner.GetFullName()	Anna Nguyen
Number of owner.OwnedProperties	1
Number of tenant.RentalContracts	1
property.IsAvailable	false
contract.IsActive	true
contract.RentAmount	5000

```
property.UpdateRentalRate(5500)
```

Expected: property.RentalRate becomes 5500, while contract.RentAmount remains 5000.

```
contract.Terminate()
```

Expected: contract.IsActive becomes false and property.IsAvailable becomes true. The contract remains in tenant.RentalContracts.